

Deliverable A – Edge Application Specification

1. Purpose, Scope, Deployment Context

The **Edge Application** (collector) runs at each PDO site to interface the local DCS OPC A&E servers and ingest alarm/event data into Kafka. It is responsible for subscribing to each OPC A&E server via **local COM/DCOM** (no network DCOM or tunnellers ¹) and streaming events northward. The collector may be deployed **co-resident** on the OPC server host or on an adjacent VM on the same secured subnet (co-located to satisfy the “local COM” requirement). *Assumption:* the PCD allows installing a Windows-based agent in either configuration. Containerisation (Docker/Kubernetes) in PCD is an open item: if disallowed, use Windows Service or similar.

The collector’s **scope** is alarm ingestion and normalization only; operator acknowledgement and shelve commands are sent back to the DCS via the collector. No UI or KPI functions are handled at the edge. It must support 74 heterogeneous sites with *multi-OEM DCSs* (Honeywell, DeltaV, Yokogawa, Foxboro, ABB, GE, etc.). All upstream transport is via **Kafka over TLS** (no remote DCOM). The target end-to-end latency is “near-real-time” (~5–10 s). Any stricter requirement (≤ 5 s) is flagged as an open item, since worst-case latencies (COM callbacks, Flink processing, network) may exceed it.

Power/space constraints in PCD mean the collector should be lightweight: minimal CPU/RAM footprint (e.g. <2 vCPU, <4 GB RAM per site) if co-resident. Adjacent-VM deployment allows slightly higher resource budgets. The service must run under a least-privilege account and be hardened (firewalled except Kafka endpoints, patched, etc.). Use TLS/mTLS for Kafka; OPC COM remains local-only.

Open items: Data-diode vs. bidirectional firewall at PCD↔IOC (determines whether write-back ACKs can flow down); exact hosting (co-resident vs. adjacent VM); PCD virtualization policy; AVEVA System Platform integration.

2. OEM Connector Abstraction

Define a plugin-based **Driver Interface** (e.g. a C++/C# class or service) with methods: `connect()`, `subscribe()`, `refresh()`, `processEvent()`, `acknowledge()`, and `disconnect()`. Concrete drivers implement this interface per OEM (plus Foxboro’s SQL driver and an optional OPC UA A&C driver). Each driver reads configuration (e.g. CLSID/ProgID, connection params, filter criteria, tagging rules) and handles OEM-specific details.

- **Honeywell Experion (Classic/TPS/TDC):** Uses the Experion OPC A&E server. Native alarm priorities: *Journal, Low, High, Urgent* ². Map these into OPC severity (1–1000) and CAS priority (e.g. ISA 1–4) in the normalization layer. The driver must use `IOPCEventServer` (COM) and supply any Honeywell-specific area/asset attributes (e.g. Area Name, Alarm Class) found on the OPC event. OPC UA A&C is available on newer Experion; the collector abstraction should allow swapping to UA (via UA Client library) if the site supports it.
- **Emerson DeltaV:** Uses the DeltaV OPC Events Server (A&E) ³ ⁴. Native priorities: *Critical, Warning, Advisory* (plus Diagnostics) by alarm definition. Map Critical→high severity, Advisory→low. The driver reads `Source` as module/parameter path, and DeltaV’s severity (0–

100 scale or bitmask for Critical/Warning/Advisory) and converts to 1–1000. DeltaV timestamps are reliable (originating from control loop).

- **Yokogawa CENTUM/ProSafe (Exaopc):** Uses Yokogawa's Exaopc OPC A&E server ⁵. Native priority levels (e.g. 1–5) should be mapped. The `Source` will be I/O point path. Exaopc provides OPC Severity 1–1000; trust it. Implement area/asset mapping using Yokogawa's tags if provided.
- **Foxboro I/A (Schneider Electric):** *No native OPC A&E server.* Instead, use FoxAPI or directly query the Fox Historian/Alarm DB. For example, poll the I/A alarm log tables or use the FoxAPI OBJECTMGR to retrieve alarm records. Timestamps often reflect **receive time** rather than true SOE. Therefore, tag all Foxboro alarms with a lower fidelity flag and **fallback**: use the ingestion time for event-time ordering. As per requirements, acknowledge Foxboro alarms via its interfaces (if possible) or mark ACK commands as complete after local event confirmation.
Timestamp issue: Foxboro's lag (if any) may challenge a strict ≤ 5 s window; consider as open if sub-5s is mandated.
- **ABB 800xA (AC800M):** Has an OPC A&E interface (and OPC UA A&C). Supports grouping/hiding. Use the OPC Classic A&E if running on legacy modules or UA A&C if allowed. ABB native priorities align with ISA; map accordingly. The driver should be pluggable to either COM or UA.
- **GE Mark VIe / Proficy iFIX/CIMPLICITY:** Use their OPC A&E servers. Mark VIe alarm severities (e.g. 1–5) map to OPC 1–1000. CIMPLICITY has alarm classes; map similarly.
- **Matrikon OPC A&E:** Skip – tunneller disallowed. (Matrikon connectors cannot be used per requirements.)

Each driver must handle the OPC event callback (`IOPCEventSink::OnEvent`), extract the OPC COM `ONEVENTSTRUCT` data, and emit a *canonical event record*. All drivers should expose a uniform interface to Flink (via Kafka) so that downstream processing is OEM-agnostic. Configuration for each driver includes: server identifier, filter criteria (areas, severity range, event types) ⁶, and normalized area/asset names.

3. Connection & Subscription Management

Each driver creates **one OPC A&E subscription** (`IOPCEventServer::CreateEventSubscription`) per OPC server (the “single string” requirement). Upon connect (and on reconnect after any failure), the driver must explicitly call `Refresh()` ⁷ to retrieve all currently active/unacknowledged conditions (building the current alarm summary). The COM subscription should specify minimal filtering on the server (e.g. limit to “Condition events” or configured areas/severity ranges ⁶) but rely mainly on edge filtering for precise needs. Use `IOPCEventSubscriptionMgt::SetFilter` judiciously: server-side filtering reduces load, but we must keep **one subscription per server** and handle all relevant events in our code.

The driver must implement robust session management: if the COM connection breaks or the OPC server goes down, the driver should automatically retry with exponential backoff, log errors, and resume subscriptions. On reconnect, call `Refresh()` again to reconcile any missed changes ⁷. Track a simple heartbeat metric (e.g. last-event timestamp per server) and alert if no events arrive for a configurable timeout.

Server-side filters (areas, categories, severity, event type) may be set via `SetFilter()` to drop irrelevant events ⁶. For example, if the site's alarm philosophy indicates only severities above a threshold or specific areas matter, configure that on subscription creation. Otherwise do filtering in the edge logic.

Each event callback passes a `ONEVENTSTRUCT` (see **Event Ingestion**). The driver must extract all fields and vendor-specific attributes (via `pEventAttributes`). For vendor custom fields (e.g. Honeywell's AlarmClass, Yokogawa's area), the driver should append them to the event record under normalized keys. The driver also captures the OPC session's `Cookie` on each event and *replaces it at ack time* – do not persist cookies, only the immutable tuple `(Source, ConditionName, ActiveTime)` ⁸.

4. Event Ingestion & Canonical Schema

Define a canonical schema (e.g. Avro or Protobuf) for **alarm/event records**. Each record represents one OPC event or tracking event. The schema fields include all OPC A&E data plus normalized fields (see example below). All ingested events are published to a local Kafka topic (keyed by a unique event ID or by `(siteId, source, conditionName, activeTime)` if unique). The main fields are:

```
AlarmEvent {
  siteId:          string; // e.g. "Site007"
  oemType:         string; // e.g. "Honeywell", "DeltaV", etc.
  serverId:        string; // OPC server identifier or DCS area
  source:          string; // OPC Event src (tag path or device)
  conditionName:   string; // OPC ConditionName (alarm point ID)
  subCondition:    string; // OPC SubConditionName (e.g. "HI", "LOLO")
  eventType:       string; // OPC event type (e.g. "Condition")
  severityRaw:     int;     // 1-1000 severity as reported 9
  severityNormalised: int; // Mapped 1-1000 severity in CAS scheme
  priorityNormalised: int; // Mapped ISA-18.2 priority (1-4) or custom
  priority
  activeTimeMs:    long;    // OPC ftActiveTime in ms since epoch 10
  eventTimeMs:     long;    // OPC ftTime (notification time) in ms since
epoch 11
  changeMask:      int;     // OPC wChangeMask bits (Active, Ack, Enable,
etc) 12
  newState:        {enabled:bool, active:bool, acked:bool}; // from wNewState
4
  ackRequired:     bool;    // OPC bAckRequired flag 10
  quality:         int;     // OPC Quality (Data Access flags)
  message:         string;  // OPC Message text
  area:            string;  // Normalised area tag (from vendor attribute)
  asset:           string;  // Normalised asset/tag hierarchy (vendor
attribute)
  cookie:          bytes;   // OPC event Cookie (for AckCondition) 13
  ingestTimeMs:    long;    // Ingestion timestamp at collector
  timestampFidelity: string; // e.g. "device" or "ingest" to indicate source
of time
}
```

All numeric times are in millisecond resolution (FILETIME truncated to ms). The **canonical schema** must be versioned and stored in a schema registry for evolution. Use `siteId` + `(source, condition, activeTime)` as a unique key for alarm instance identity.

Upon receiving a raw OPC event, the driver populates this schema:

- `severityRaw` is copied directly; `severityNormalised` and `priorityNormalised` are computed per-OEM (see Section 5).
- `activeTimeMs` is OPC's reported activation time (`ftActiveTime`), which is the SOE timestamp ¹⁰. If the OEM cannot supply true SOE (e.g. Foxboro), set `activeTimeMs` = arrival time and mark `timestampFidelity` = "ingest".
- `eventTimeMs` is `ftTime` (notification time) ¹¹.
- `changeMask` and `newState` come from `wChangeMask` and `wNewState` bits ¹² ⁴.
- `cookie` is carried through (but not persisted beyond this record).

Each event record is serialized (e.g. Avro/Protobuf) and produced into a **Kafka topic** (e.g. `Site007.raw-events`). The key should include the instance ID (`siteId`, `source`, `conditionName`, `activeTimeMs`) to allow partitioning and state management. This raw topic is retained to support store-and-forward; retention should cover expected maximum offline interval (e.g. days to weeks).

5. Normalisation at the Edge

At ingestion time, the collector converts vendor-specific values into the CAS canonical scheme:

- **Severity/Priority Mapping:** For each OEM, define a table mapping native priority levels to OPC 1–1000 and to ISA-style priorities. For example, Honeywell's *Journal/Low/High/Urgent* might map to OPC severity 100/400/700/1000, and to ISA priority 4/3/2/1 respectively ². DeltaV's *Advisory/Warning/Critical* map similarly (Advisory≈low, Critical≈high). Yokogawa's numeric priority bands map into 1–1000 linearly. Define these mappings in configuration.
- **Area/Asset Normalisation:** Extract vendor "area" or "unit" attributes (e.g. Experion Area, DeltaV Area, Yokogawa Unit) and map them to a common `area` field. Similarly map any asset or unit identifier to `asset`. If vendor provides hierarchical tags, flatten or concatenate (e.g. `Area1/Unit5`). Missing fields default to empty.
- **Message Normalisation:** The `message` text may include codes or parameter values; retain it verbatim. Remove any binary non-text attributes.
- **Timestamps:** As noted, use controller-provided time if available; otherwise use ingest time and set `timestampFidelity="ingest"`. Store both `activeTimeMs` (the stable SOE anchor) and `eventTimeMs` (callback receipt time) in the record for later ordering.
- **Watermark Assignment (for Flink):** The edge Flink job (Section B5) will use `activeTimeMs` as event time with bounded out-of-orderness. At ingest, annotate each event with a "fidelity" flag to let Flink decide if it can be used as watermark basis (if `timestampFidelity="device"`) or if it should be ignored or held (if low fidelity).

After normalization, the driver **publishes** the normalized event record to an edge Kafka topic (e.g. `Site007.normalised-events`), partitioned by `siteId` and perhaps hash(source). It also forwards

the raw event (for completeness) if raw and normalized differ; otherwise raw/normalized can be the same record. The normalized topic uses short retention (just long enough for transport to IOC, e.g. 24–48 h).

6. Store-and-Forward & Backfill

Each site has a **local Kafka broker** (or cluster). The raw-events topic (and normalised-events, alarm-state, etc.) use Kafka's retention to buffer data during upstream outages. For example, retention should exceed the longest anticipated IOC unavailability (e.g. hours/days). This ensures *zero alarm loss*: when the IOC link is down, Edge Flink continues processing locally.

On reconnect, Kafka's retained log is replayed. Use Flink's **event-time processing** (with watermarks) so that replayed (late) events still integrate correctly into the site's state. The Flink job will have logic to allow a bounded lateness (e.g. up to a few minutes) and side-output any excessively late events for analysis.

The collector ensures ordering by writing events to Kafka in the order received. The `activeTimeMs` embedded in each event allows Flink to restore SOE order across sites when we replay.

Sizing: The site's Kafka broker must allocate disk for retention based on worst-case alarm flood rates and retention period. For example, if a site can produce 1000 alarm events/sec (worst-case flood) and we retain 24 h, that's ~86.4M events; at ~1kB each, ~80 GB. This should be computed per site (open item: actual expected flood rate, retention targets).

7. Acknowledgement / Command Handling (Write-Back)

The collector subscribes to a local Kafka **commands** topic (e.g. `Site007.cmd-ack`) for operator actions from IOC. Messages include: `{siteId, source, conditionName, activeTimeMs, commandType, commandId, timestamp}`. When a command is received:

- **Ack Command:** On receiving an ACK command, the collector looks up the corresponding OPC Cookie by matching `(source, conditionName, activeTime)` against a cached or queried `IOPCEventServer.GetConditionState` call. (The `cookie` in events is session-bound, so it must be resolved live.) Then call `IOPCEventServer::AckCondition(source, conditionName, activeTime, cookie)`. Upon success, log and (optionally) publish an “ack-command-sent” event locally. If the DCS is disconnected or not acknowledging, the command should be retried or logged as error. Command idempotency is ensured by client-assigned unique `commandId` (ignore duplicates).
- **Shelve/Suppress Command:** If shelving or disabling is supported, similar pattern: translate to the DCS's mechanism (e.g. DeltaV's shelving APIs). These commands also originate from the IOC and flow to the collector, then to the DCS via the appropriate API. The collector may not implement shelf logic itself (Flink handles the conceptual state); it merely executes operator shelve/cancel commands on the DCS.
- **Command Flow Guarantee:** Because of potential one-way connectivity, the UI should not assume an ACK is completed until the DCS emits a new event with the `ACKED` bit. Therefore the UI must wait for the OPC server's ACK confirmation. The collector does **not** update UI state

optimistically; it only re-sends confirmations (via the forward Kafka path) when the OPC server reports the ACK.

Commands are published by the edge (after execution) into the command-topic for auditing (and for IOC Flink to track pending acks). Use `commandId` for idempotence.

8. Security

- **OPC Security:** The edge service communicates to OPC servers only via local COM; no DCOM opens ports, so no external OPC access. Disable network DCOM entirely.
- **Kafka Security:** Use TLS encryption and mTLS certificates on all Kafka brokers (Edge and IOC). Use Kafka ACLs to restrict producers/consumers by service account. Each edge service should have its own Kafka principal with write access only to its site's topics and read access only to its inbound commands.
- **Service Accounts:** The collector should run as a low-rights user on Windows, only with privileges needed for COM/OPC and network Kafka.
- **Hardening:** Whitelist executables in AV/Endpoint Protection to allow the collector's binaries and Kafka clients. Ensure the Windows instance is patched. Restrict network firewall: allow only required ports (OPC via localhost COM, Kafka TLS out to IOC cluster, no inbound from outside except maybe Management).

9. Resource & Footprint

Estimate per-site collector footprint (co-resident): about 1–2 vCPUs and 2–4 GB RAM should suffice under normal load. An adjacent VM can provision more if needed. Kafka broker on site: if single-node, allocate at least 100 GB disk for retention (size = event rate × retention). RocksDB state from Flink requires disk space (see section B9). The CPU load is bursty (on alarms); idle otherwise. Edge Flink job: allocate parallelism ~1–2 and memory ~4–8 GB.

If co-resident, resource budget must fit on DCS hardware (often few-core servers). If an adjacent VM is used, resources can be dedicated. Monitor actual usage during testing and adjust.

10. Configuration & Lifecycle

Centralize configuration in a version-controlled repository or config management system. Each site's config includes DCS server list, OPC ProgIDs, filter settings, normalisation rules. The collector can auto-discover new OPC tags if needed via OPC DA browse, but major new alarms should be handled in DCS configuration and then flow through automatically via `Refresh()`. Versioning: use a config schema with semantic versioning. Upgrades: use Kafka and Flink savepoints; collector upgrades can be done per-site in maintenance windows.

Open item: How are new alarms discovered? We assume OPC servers always list all conditions, so the collector just sees them on `Refresh()` or on event. No manual tag list needed.

11. Observability

Expose Prometheus-style metrics: per-OPC-server events/sec, COM connection status, Refresh latencies, Kafka produce/consumer lag. Log to local file (rotate daily) and to a central log aggregator. Tracing: instrument critical paths (subscription calls, ack calls). Alerts: fire if any OPC server is disconnected >X minutes, or Kafka backlog exceeds threshold. Also emit count of events dropped or errors.

Local dashboards (or IOC dashboards) should show each site's connectivity (OK/NOT), event rate, and per-server last event time.

12. Failure Modes & Test Plan

- **OPC Server Down:** Collector loses connection and retries. Edge Flink should detect lack of events and mark alarms stale if no refresh occurs. On server restart, collector re-subscribes and issues `Refresh()`, Flink reconciles state (see B10).
- **Network (PCD↔IOC) Down:** Kafka local buffer fills up to retention limit; edge Flink continues normal operation autonomously. When network recovers, data replicates to IOC.
- **Edge Restart:** On restart, collector re-subscribes and `Refresh()` gets current active alarms (ensuring no missing clears); then it consumes from Kafka from last committed offset to catch up.
- **Partial OEM Outage:** If one OEM's OPC server hangs, collector continues other servers. Flink state for that site still valid for other alarms.
- **Backfill Test:** Simulate IOC outage, then flood X alarms, then restore. Verify all alarms appear at IOC in correct SOE order.
- **FAT:** Build a test harness simulating OPC A&E servers (or use DCS test mode). Generate alarms at known times, including duplicates and chattering. Verify edge and IOC processing (state machine, exactly-once, no duplicates).
- **Security Test:** Verify no COM over network, Kafka TLS enforced, service runs without admin rights.

Deliverable B – Flink Alarm-Lifecycle State-Management Specification

1. Two-tier Responsibility Split

Edge Flink (Per-Site): Processes raw/normalized events *for that site only*. Responsibilities: apply OEM-specific mappings (completed by collector), assign event-time/watermarks per source, deduplicate (drop repeated identical notifications based on `changeMask`), detect local chatter/flood and mark affected alarms, and maintain a *site-local alarm state machine* (first-line lifecycle). It emits "site alarm state changes" (activations, clears, acked transitions) and computes local KPIs (e.g. site alarm rate) for local dashboards. While the IOC is disconnected, edge Flink state is authoritative for that site.

IOC Flink (Enterprise Level): Aggregates all sites. It ingests replicated normalized events and (optionally) site-state events. Its responsibilities: maintain the **authoritative consolidated alarm state** across sites, correlate alarms (e.g. same physical asset alarms), compute ISA-18.2/EEMUA metrics (operator workload, flood %, standing alarms, etc.), and serve state for the UI. IOC Flink also orchestrates command routing: it takes UI commands, forwards them to the correct site's command

topic, and only finalizes state when the ACK confirmation returns from the DCS. It emits the *log-compacted current-alarm snapshot* topic for the UI gateway and writes history to a sink (e.g. SQL or PI).

State Authority: During normal operation, IOC Flink is source-of-truth for the global view, but edge state is truth for site-local decisions when disconnected. On reconnect, edge state is reconciled to IOC state by re-injecting any missed events (via Kafka backfill) – see B10.

2. Job Topology

Each tier may run one Flink *job*, or a few dedicated jobs. For simplicity, use **one job per tier** with logical subgraphs. Edge Flink job: Source = Kafka consumer on `site.raw-events`; then `keyBy(alarmInstanceKey)` into a `ProcessFunction` implementing the lifecycle state machine. Side-output chatter/flood alerts. Sink: Kafka to `site.state-changes`, and optionally `site.current-state`. IOC Flink job: Source = Kafka on all sites' normalised-events (or on replicated `state-changes`), `keyBy((siteId, source, conditionName))`; state machine in a `KeyedProcessFunction` merges cross-site logic; compute cross-site metrics; sinks: 1) to log-compacted current-state topic, 2) to history store, 3) to dashboards. Commands: a separate Kafka source for `commands` stream, keyed by `(siteId, source, conditionName)`, routed to state processor.

A unified job is feasible, but separating edge vs IOC allows scaling and loose coupling; however, each can run identical code with different parameters.

3. Keying

The **alarm instance key** is `(siteId, source, conditionName)`. This uniquely identifies an alarm definition in the plant. Using this key groups all events of one alarm to the same partition and state. SiteId distinguishes site scope. The key cardinality is number of unique alarms per site ($O(10^4-10^5)$). Kafka topics partition by site (and key) for parallelism. Flink parallelism should match Kafka partitions to avoid skew. Use consistent hashing on this key for partition.

4. State Primitives

Use Flink keyed state:

- **ValueState<AlarmState>**: The current state of the alarm instance, containing fields: `{active:boolean, acked:boolean, shelvedUntil:timestamp, suppressed:boolean, lastChangeTime:long, priority:int, severity:int, subCondition:string, occurrenceCount:int, chatterCount:int, ...}`. Also store original `activeTimeMs` for the active cycle and any pending ack command ID.
- **ListState<EventRecord>**: (Optional) history of recent events for this alarm (e.g. last N subcondition changes) to support chatter detection or windowed metrics.
- **MapState<String, Long> chatterWindow**: For counting flaps, store timestamps of recent transitions.
- **AggregatingState** or **ValueState<Long>** for counters/metrics (e.g. count of transitions in window).

These states persist in RocksDB for durability.

5. Event-Time & Watermarks

Assign event-time from the alarm's `activeTimeMs` (which is the controller SOE) ¹⁰. Use `WatermarkStrategy.forBoundedOutOfOrderness(Duration.ofSeconds(X))` where X is chosen based on network jitter (e.g. 10s). Mark an event-time *idleness* per key after no events for some time to avoid stagnation. For low-fidelity sources (Foxboro), use ingestion time but emit a watermark that lags (i.e. treat watermark = current processing time minus a large bound). Propagate a `timestampFidelity` flag through Flink so sinks know which events have real-time stamps vs not.

6. The Transition Function (Alarm Lifecycle)

Implement a `KeyedProcessFunction` on `(site, source, conditionName)` that consumes condition-related events. Maintain the current `AlarmState`. On each event, use `wChangeMask` to apply state changes only when meaningful. The high-level state machine (ISA/EEMUA model):

States:

- **Normal:** (inactive, acked, enabled).
- **ActiveUnacked:** (active, unacked).
- **ActiveAacked:** (active, acked).
- **RTNUnacked:** (return-to-normal, unacked, i.e. latched clear).
- *Auxiliary flags:* Shelved, Suppressed, Out-of-Service. (Handled orthogonally: e.g. if a shelve command is active, treat alarms as suppressed until expire.)

Transitions (Event × State → New State + Action):

Current State	Event	New State	Action
Normal	ConditionActive (ACKRequired)	ActiveUnacked	Emit "Alarm Activated" event (unacked), increment count.
Normal	ConditionActive (ACKNotRequired)	ActiveAacked	Emit "Alarm Activated" (auto-acked).
ActiveUnacked	Ack command received	ActiveAacked	Emit "Alarm Acknowledged".
ActiveUnacked	ConditionInactive	RTNUnacked	Emit "Alarm Cleared (latched)".
ActiveAacked	ConditionInactive	Normal (inactive)	Emit "Alarm Cleared (cleared)".
RTNUnacked	Ack command received	Normal	Emit "Alarm Acknowledged (after RTN)".
RTNUnacked	ConditionActive (again)	ActiveUnacked	Re-activated; emit "Alarm Re-activated".
<i>Any Active</i>	Enable=false (disable)	OutOfService	(Emit "Disabled" event)
<i>OutOfService</i>	Enable=true	(return to prev)	(Re-enable to previous state)

Current State	Event	New State	Action
<i>Any (enabled)</i>	Shelve command	Shelved	(Emit “Shelved” state; schedule timer to unshelve)
<i>Shelved</i>	Shelve expires	(restore prev state)	(Unshelve automatically)

Only emit a state-change record if `wChangeMask` indicates a relevant bit (Active, Ack, Enable) changed in the OPC event ¹². For example, if we receive an OPC callback with `ACTIVE_STATE` and `ACK_STATE` changed, handle both effects. If multiple events coalesce, handle in order. Always latch `RTNUnacked` until ack arrives.

Use `processElement()` to update state and emit a normalized state-change record (schema below).

7. Timers

- **Event-time timers:** Implement off-delay or debounce if needed (e.g. if hardware reports flapping on spike, wait a few ms before final state). Also may use event-time timers to emit KPI intervals (though IOC does that).
- **Processing-time timers:** For auto-unshelve (expire shelve after a timeout) and to clear stale state. For example, if an alarm remains active but no event seen for a long time, one might clear it (depending on design).
- For *standing alarm cleanup*: if an alarm stays active but stale (no corresponding process data), a timer could re-query or clear.

Timers can also schedule window rolls for chatter detection: e.g. after 1-minute window, collapse chatter count.

8. CEP / Flood & Bad-Actor Logic

- **Chatter Detection:** Maintain a sliding count of how many times this alarm state flipped (Active→Inactive→Active, etc.) within a short time window. Use Flink’s CEP or just keyed state with timers. If count > N in T seconds (configurable, e.g. 3 flips in 10s), emit a “chatter” event on a side-output for that alarm, and optionally suppress emitting repeated notifications (e.g. throttle further active events for a short quarantine).
- **Flood Detection:** Count total alarms (across conditions) per site/area in sliding window. If alarm rate > threshold (EEMUA suggests e.g. >10/min/operator), flag an “alarm flood” to raise an alert or trigger load-shedding.
- **Bad-actor Ranking:** Maintain per-alarm counters (all-time or sliding) of activations. Periodically output metrics (e.g. top 10 alarms by count) to help rationalization.

These analyses can be side outputs to monitoring topics or a metrics store.

9. Exactly-Once & State Backend

Configure Flink for **checkpointing** at e.g. 30s intervals. Use the **RocksDB** state backend (incremental checkpoints) on a durable storage (e.g. on-prem object store or HDFS/NFS) to handle OT requirements. Enable Kafka checkpointing: the `KafkaConsumer` should commit offsets on checkpoint. Use a Kafka Producer with **transactions** (the `FlinkKafkaProducer` in `EXACTLY_ONCE` mode) for all sinks ¹⁴. For

example, writing to the log-compacted topic must be transactional so that each checkpoint corresponds to one commit in Kafka. This ensures end-to-end exactly-once ¹⁴ even in failures.

Configure Kafka sink as:

```
KafkaProducer<String, AlarmState> withTransactionalId("FlinkIOC")
```

so each checkpoint is a transaction. If a job fails after checkpoint, upon restart no duplicate writes occur. Commands (ack send) should also be idempotent; correlate `commandId`s so that retries do not double-ack.

10. Recovery & Refresh() Reconciliation

On restart, Flink restores from the last checkpoint (state is recovered). Immediately after startup, the collector drivers will issue OPC `Refresh()` calls, generating a burst of “refreshed” events ⁷. These must be processed by Flink to reconcile any changes that occurred during downtime. For example, if an alarm was acked while Flink was down, the refresh event will show it active+acked or inactive. The transition function should compare restored state vs incoming refresh state: any difference should be emitted as a state-change event. For example, if Flink thought an alarm was `ActiveUnacked` but refresh comes in as `Inactive` (implying `Clear+ACK` happened), then Flink should move to `Normal` and emit `Clear+Ack` events. Treat refresh events like normal events, but note they may come out-of-order.

Rule: If refresh timestamp < current state lastChangeTime, ignore as stale. Otherwise, apply transitions normally.

11. Backfill Ordering

Because Kafka partitioning by (site,key) and Flink event-time, backfilled events will be ordered into proper SOE order by the watermark strategy. Set a maximum lateness (e.g. 1 min) so late events are still processed; any events beyond lateness go to a “late” side output topic rather than be dropped. The result is a correct millisecond-resolution SOE ordering, satisfying the fidelity requirement.

12. Serving the Current State

Maintain a **log-compacted** Kafka topic (e.g. `alarm-current-state`) keyed by `(siteId,source,conditionName)`. Upon each state change (active, ack, clear), output a message with the new full alarm state (active/inactive, acked/unacked, severity, priority, lastChangeTime, etc.) under that key. Because it is compacted, the topic always retains the latest state per alarm. A WebSocket/SSE gateway can tail this topic to push updates to HTML5 clients. *Do not* rely on Flink Queryable State for UI (it is optional) – instead use Kafka. Also sink a full history of alarm-state-change records to a time-series database or historian (e.g. InfluxDB, OSI PI) for audit.

Schema for `AlarmState` in compacted topic:

```
AlarmState {  
  siteId, source, conditionName, subCondition,  
  active: bool, acked: bool, enabled: bool,
```

```

priority: int, severity: int,
lastActiveTimeMs, lastAckTimeMs, lastClearTimeMs,
stateChangeTimeMs, // when state last changed
shelvedUntilMs, suppressed:bool
}

```

Publish on every transition. The topic uses key = `(siteId,source,conditionName)`.

13. Command / Ack Orchestration

In IOC Flink, subscribe to the consolidated commands topic (all UI-generated ack/shelve commands). Key by `(siteId,source,conditionName)` and correlate against current state. For an ACK command, mark the alarm state as *Ack-Pending* (perhaps set a flag in state). Produce the command onto the edge's Kafka command topic (routed by siteId). Only when the corresponding condition event with `ACKED=true` flows back up from the edge (via normal event flow) do we mark the pending as completed and update the state to acked. This ensures the UI only sees the ACK when the DCS has confirmed it. Use the command's unique ID for idempotency (ignore duplicate commands). All commands (and their outcomes) should be logged for audit.

For shelving commands, simply update the state and pass the command down, then clear state when the shelve expiry event returns.

14. KPIs & Analytics

Calculate ISA-18.2/EEMUA KPIs in IOC Flink (or export to a monitoring system):

- **Alarm Rate:** count alarms/operator/min. (From ISA: aim <1–2 alarms/10 min ¹⁵ ; alert if above).
- **Standing Alarms:** count of alarms active > X hours (per EEMUA guidance) ¹⁵ .
- **Flood Percentage:** fraction of alarms during flood conditions.
- **Nuisance/Repeater Alarms:** list alarms that exhibit chattering or high recurrence.
- **Operator Response:** distribution of Ack times (time from event to ACK).

Emit these metrics periodically (via Prometheus, Grafana, or push to an analytics DB). Also maintain SLA alerts (e.g. if avg ack time > Y, etc.).

15. Schemas

Alarm Event Record: (as in Section A4).

Alarm-State-Change Record: `{siteId,source,conditionName, subCondition, oldState, newState, eventTimeMs, ackTimeMs?, severity, priority, eventType, changeMask}`.

Command Record: `{commandId, siteId, source, conditionName, activeTimeMs, commandType (ACK/Shelve/etc), operatorId, timestamp}`.

Current Alarm State Record (compacted): (as described in §12).

All schemas use stable keys (as above) and are versioned. The topic keys are specified in the Kafka Topic Catalogue (below).

16. Scaling, HA & Sizing

- **Parallelism:** Flink parallelism must match Kafka partitions (e.g. 4–8). Key cardinality per partition (~100k alarms) is manageable with RocksDB.
- **Checkpointing:** Store to high-availability storage (e.g. HDFS/MinIO) with replication.
- **Throughput:** Input sizing = peak alarm rate across all sites (e.g. assume 1000 events/sec/site * 74 sites = 74k events/s). Ensure cluster can handle bursts.
- **State Size:** Estimate state entries = total configured alarms (maybe 100k/site active config, 74 sites → 7.4M keys worst-case) but active subset is smaller. Use a cluster of several Flink TaskManagers (e.g. 10–20 nodes) for headroom.
- **Topic Partitions:** IOC topics partitioned by site then by alarm key, to spread load.
- **HA:** Run Flink in cluster mode (YARN/K8s), with standby JobManagers. Kafka cluster should be multi-broker with replication across zones.

17. Test & Verification

- **Unit Tests:** For the state machine logic (all transitions), including edge cases (e.g. double-acks, disable/enable during active).
- **Integration Tests:** Simulate OPC event streams (including out-of-order and duplicate events) and verify final state.
- **Outage Tests:** Simulate IOC or Edge failures; verify that on recovery, the combination of Kafka retention and `Refresh()` leads to correct state.
- **Exactly-Once Test:** Kill Flink tasks mid-processing; after recovery, ensure no lost or duplicated events (via comparing counts).
- **Chatter/Flood Tests:** Inject rapid flapping alarms and bursts to confirm detection logic triggers.
- **Performance Test:** Load-test with synthetic alarm floods at or above expected peak to size the Kafka retention and Flink cluster.
- **End-to-End Test:** From DCS to UI: generate alarms on DCS, observe them on web banner, perform ack, and confirm UI updates only after DCS echo.

Kafka Topic Catalogue (example)

Topic Name	Key	Partitions	Retention / Compaction	Producers	Consumers
<code>siteX.raw-events</code>	<code><siteId>:<source>:<cond>:<time></code>	1 (per site)	7d retention	OPC Collector (edge)	Edge (state machine)
<code>siteX.normalised-events</code>	same as raw	1	24h or 7d	Edge Flink (normaliser)	IOC (agg)
<code>siteX.state-changes</code>	<code><siteId>:<source>:<cond></code>	1	7d	Edge Flink (state machine)	IOC (agg)

Topic Name	Key	Partitions	Retention / Compaction	Producers	Consumers
<code>siteX.cmd-ack</code>	<code><siteId>: <cmdId></code>	1	Compact (retain last status)	IOC Flink (router)	OPC Collector (executable)
<code>alarm-current-state</code>	<code><siteId>: <source>: <cond></code>	(multisite) e.g. 74	Log-compacted (infinite)	IOC Flink (state output)	UI Gateway (Web service)
<code>alarm-history</code>	e.g. auto-increment or UUID	e.g. 10 partitions	Time-based (years)	IOC Flink (history sink)	BI/Analytics system (SQL, etc.)

Partitions per topic should match expected parallelism (per-site topics can be single-partition if colocated with single Flink instance; global topics may have many partitions).

Per-OEM Normalisation Table (example excerpt)

OEM	Native Priority Levels	OPC Severity 1–1000 Mapping	CAS Priority (ISA 1–4)	Timestamp Source	Ack Path
Honeywell	Journal, Low, High, Urgent ²	100, 400, 700, 1000	4, 3, 2, 1	Controller time	OPC.AckCond
Emerson ΔV	Advisory, Warning, Critical	200, 600, 1000	4, 2, 1	Controller time	OPC.AckCond
Yokogawa	1–5 (VP)	linear to 1–1000	4→1	Controller time	OPC.AckCond
Foxboro	1–5	linear to 1–1000 (approx)	4→1	Often receive-time*	FoxAPI or none*
ABB 800xA	1–5	linear 100–1000	4→1	Controller time	OPC.AckCond
GE MarkVIe	1–5	linear to 1–1000	4→1	Controller time	OPC.AckCond

Open item: Foxboro timestamp is not true SOE (usually DCS receive-time), so we treat it as lower fidelity. Foxboro ack path may require its proprietary API; verify at each site.

Each mapping must be vetted with the site's alarm philosophy (ISA-18.2 style).

State-Machine Transition Table (example)

Old State	Event (changeMask bit)	New State	Actions
Normal	ActiveState +AckReq	Active, Unacked	Emit Alarm On; start timer (chatter)
Normal	ActiveState (no Ack)	Active, Acked	Emit Alarm On (auto-ack)
Active, Unacked	AckState	Active, Acked	Emit Alarm Acked
Active, Unacked	ActiveState false	RTN, Unacked	Emit Alarm RTN (latched)
Active, Acked	ActiveState false	Normal (Inactive)	Emit Alarm Cleared
RTN, Unacked	AckState	Normal (Inactive)	Emit RTN Acknowledged → effectively clear
<i>Any</i>	Enable=false (DISABLE)	Out-of-Service	Emit Disabled
Out-of-Service	Enable=true	(prev state)	Emit Re-enabled
<i>Any (ENABLED)</i>	ShelveCmd	Shelved	Emit Shelved; schedule unshelve
Shelved	ShelveTimer expires	(prev state)	Emit Unshelved

Only transitions triggered by OPC bits (`wNewState`) and operator commands occur; ignore unchanged events.

Note: This table omits suppressed-by-design transitions (handled externally by logic) and assumes no simultaneous multi-bit flips (treat each flag independently).

1 6 OPC A&E

<https://atdocs.inmation.com/system/1.110/opc-connectivity/opc-a-e.html>

2 Experion LX HMI Specification

<https://prod-edam.honeywell.com/content/dam/honeywell-edam/pmt/hps/products/pmc/field-instruments/pmt-hps-lx03-200-520-experion-lx-v01-0.pdf>

3 DeltaV OPC Events Server

<https://www.emerson.com/documents/automation/product-data-sheet-deltav-opc-events-server-en-56268.pdf>

4 7 8 9 10 11 12 13 Alarms and Events Custom Interface

<http://advosol.com/OpSpecs/OPC%20AE%201.10%20Specification.pdf>

5 GS 36J02A10-01E

<https://library.yokogawa.com/document/download/AxkHnJ89/0000005914//EN/>

14 An Overview of End-to-End Exactly-Once Processing in Apache Flink (with Apache Kafka, too!) | Apache Flink

<https://flink.apache.org/2018/02/28/an-overview-of-end-to-end-exactly-once-processing-in-apache-flink-with-apache-kafka-too/>

15 Alarm Help

<https://www.emerson.com/documents/automation/white-paper-alarm-help-deltav-en-57056.pdf>